Methods

activity_main.xml — layout with button and textview

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:gravity="center">

    <!– Shows results from our methods –>
    <TextView
    android:id="@+id/tvResult"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:layout_marginBottom="24dp"
/>

<!– Button that calls all our methods when tapped –>
<Button
    android:id="@+id/btnRun"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Run Methods"
/>
```

```
</LinearLayout>
```

MainActivity.kt — all method types in one file

```
// ─── IMPORTS ────────────────────────────────────────────────────────────

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

// ─── CLASS ──────────────────────────────────────────────────────────────

class MainActivity : AppCompatActivity() {

    // ══════════════════════════════════════════════════════════════════
    // METHOD ZONE — all methods live INSIDE the class but OUTSIDE onCreate
    // ══════════════════════════════════════════════════════════════════


    // ── TYPE 1: Method with NO parameters and NO return value ──────────
    // "fun" declares a function/method
    // "showWelcome" is the name we give it — we choose this name
    // "()" means it takes no input
    // no return type written = returns Unit (nothing), same as void in Java
    fun showWelcome() {
        // this method just shows a Toast — no input needed, gives nothing back
```

```
    Toast.makeText(this, "Welcome to the app!", Toast.LENGTH_SHORT).show()
}
    // to CALL this method: showWelcome()


    // ── TYPE 2: Method WITH parameters, NO return value ──────────────────────
    // "name: String" = this method needs a String value passed in
    // "age: Int"     = it also needs an Int value passed in
    // you can have as many parameters as you need, separated by commas
    fun greetStudent(name: String, age: Int) {
        // use the parameters like normal variables inside the method
        // $name and $age are replaced with whatever values were passed in
    Toast.makeText(this, "Hello $name, you are $age years old!",
Toast.LENGTH_SHORT).show()
}
    // to CALL this method: greetStudent("Mpho", 21)


    // ── TYPE 3: Method with NO parameters but WITH a return value ────────────
    // ": String" after the () tells Kotlin this method RETURNS a String
    // common return types: String, Int, Double, Boolean
    fun getAppName(): String {
        // "return" sends this value back to wherever the method was called
        // the method stops running as soon as it hits return
        return "Horizon AI App"
}
    // to CALL: val name = getAppName()  → name now holds "Horizon AI App"
```

```
    // ── TYPE 4: Method WITH parameters AND a return value ──────────────────

    // takes two Int values, adds them, and returns the result as an Int
    fun addNumbers(a: Int, b: Int): Int {
        val total = a + b  // do the calculation using the parameters
        return total       // send the result back to where method was called
    }
    // to CALL: val result = addNumbers(10, 5)  → result = 15


    // ── TYPE 5: Method WITH parameters AND a String return value ────────────
    // real world example — works out a grade from a score
    fun getGrade(score: Int): String {
        // when as expression inside a method — returns a value
        return when {
            score >= 90 -> "A - Distinction"   // if score is 90 or above
            score >= 75 -> "B - Merit"         // if score is 75 to 89
            score >= 60 -> "C - Pass"          // if score is 60 to 74
            else        -> "F - Fail"          // anything below 60
        }
    }
    // to CALL: val grade = getGrade(85)  → grade = "B - Merit"


    // ── TYPE 6: Method that returns Boolean (true or false) ─────────────────
    // useful for checking conditions — returns true or false
```

```
    fun isAdult(age: Int): Boolean {
        return age >= 18  // returns true if age is 18 or more, false if not
}
    // to CALL: val adult = isAdult(21)  → adult = true


    // ——— onCreate() — where we CALL all the methods
_____________________________________

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

        // connect to UI widgets
        val tvResult = findViewById<TextView>(R.id.tvResult)
        val btnRun   = findViewById<Button>(R.id.btnRun)

    btnRun.setOnClickListener {

            // ── CALLING METHODS
_____________________________________________________

            // call Type 1 — no params, no return, just runs
    showWelcome()

            // call Type 2 — pass in a name and age
    greetStudent("Mpho", 21)

            // call Type 3 — no params, store the returned String in a variable
```

```
val appName = getAppName()
// appName now holds "Horizon AI App"

// call Type 4 — pass two numbers, get the sum back
val sum = addNumbers(10, 5)
// sum now holds 15

// call Type 5 — pass a score, get a grade String back
val grade = getGrade(85)
// grade now holds "B - Merit"

// call Type 6 — pass an age, get true/false back
val adult = isAdult(21)
// adult now holds true
```

```
            // build a result string using all the returned values
            // \n = new line
            val output = "App: $appName\n" +
              "Sum: $sum\n" +
              "Grade: $grade\n" +
              "Is Adult: $adult"

            // display all results in the TextView on screen
        tvResult.text = output
    }
}
```

```
} // end of class
```

The most important rule about methods — where they go in the file:

```
class MainActivity {

    fun myMethod() { }   ← methods go HERE (inside class, outside onCreate)

    override fun onCreate() {
        myMethod()       ← you CALL them here inside onCreate
    }
}
```